Cost-Latency Tradeoff in LLM-Based Systems: A Case Study of Prompting, RAG, and Context Optimization Across API and Resource-Constrained Local Deployment

Buddhima Sandaru Senarathna
Department of Computer Science
University of Helsinki
Helsinki, Finland
buddhima.senarathna@helsinki.fi

***Abstract*—Large Language Models (LLMs) face significant cost and latency challenges in production deployments, particularly for long regulatory documents. This paper compares three architectures–prompt-only, Retrieval-Augmented Generation (RAG), and context-optimized systems–across cloud API (Gemini 2.5 Flash) and local open-weight (Qwen 2.5 1.5B) deployments using European Union (EU) regulatory question and answering (QA) tasks.**

Results show prompt-only systems are infeasible for local deployment (0% Exact Match (EM), 67 s latency) and costly for API use (79k tokens/query). RAG achieves the lowest latency (3.3 s) and token usage (785–2.7k total tokens/query) across both settings, but yields lower Exact Match (EM) under a strict extractive evaluation. Context-optimized systems provide the best trade-off for API deployment, matching prompt-only EM (80%) with 40 % fewer tokens.

The findings demonstrate no universal optimum: RAG excels in resource-constrained local settings while context optimization balances accuracy and cost for API deployments. These results provide practical engineering guidelines for LLM system design beyond accuracy alone, emphasizing cost, latency, and hardware feasibility as first-class objectives.

***Index Terms*—Retrieval-Augmented Generation, context optimization, LLM deployment, cost-latency tradeoff, regulatory QA**

I. Introduction

Large Language Models (LLMs) have become central to natural language processing systems, enabling document-based question answering, summarization, and conversational agents. While recent advances focus on improving accuracy and reasoning, practical deployment introduces challenges related to *computational cost*, *latency*, and *resource utilization*. In production settings, inference costs scale with token usage and response latency directly impacts user experience [1], [2]. Modern deployments increasingly rely on both cloud-based API models and locally hosted open-weight models, each with distinct cost, scalability, and latency trade-offs.

Retrieval-Augmented Generation (RAG) improves LLM performance on knowledge-intensive tasks by retrieving relevant documents from external knowledge bases [3]. RAG enhances factual accuracy and reduces hallucinations compared to prompt-only approaches [4], but introduces embedding models, vector databases, and retrieval pipelines that add computational overhead [5].

Despite extensive RAG research, *there is limited empirical analysis comparing prompt-only, RAG-based, and simpler context-optimized approaches from a cost-latency perspective*. Prompt-only systems embed all information directly into prompts (minimal overhead but high token usage for long documents), while RAG trades complexity for grounding, and emerging long-context LLMs question retrieval necessity [7], [8].

This paper investigates how *LLM* system designs trade off cost, latency, and quality under realistic deployment constraints, comparing: (1) prompt-only, (2) RAG-based, and (3) context-optimized systems across an API-based frontier model and a resource-constrained local open-weight model. The goal is to compare practical deployment configurations rather than to isolate model-serving effects under matched model capacity. We evaluate token usage, end-to-end latency, and output quality on regulatory QA tasks across API and local deployments. Our findings show RAG improves quality in some settings but context-optimized systems often achieve competitive performance with lower cost and latency, providing practical guidelines for efficient LLM system design beyond accuracy alone.

II. System Design and Experimental Setup

This chapter presents the system architectures, language models, corpus construction, and evaluation protocol used in this study. The objective is to compare alternative long-context processing strategies for regulatory question answering under realistic deployment constraints.

A. Large Language Models

To reflect realistic deployment scenarios, this study evaluates two fundamentally different categories of LLMs:

- A **cloud-based API model**
- A **locally deployed open-weight model**

This distinction enables analysis not only of architectural design choices (prompt-only, RAG, context optimization), but also of deployment trade-offs between external inference services and on-device computation.

1) Gemini 2.5 Flash (API-based Model): The first model used was `Gemini 2.5 Flash`, accessed via a cloud-based API. This model represents a highly optimized, production-grade LLM designed for low-latency inference and efficient token usage. As an API-based model, it benefits from large-scale infrastructure optimization but introduces per-token cost and network latency.

2) Qwen 2.5 1.5B Instruct: The second model was `Qwen/Qwen2.5-1.5B-Instruct`, a 1.5B-parameter instruction-tuned decoder-only transformer with a 32,000-token context window. This model was deployed locally on GPU hardware, avoiding per-token API cost but remaining constrained by GPU memory and processing speed.

3) Comparison Rationale: The inclusion of both models enables a realistic comparison between two deployment profiles commonly faced by practitioners:

- **API-based inference**: scalable, optimized, but cost-incurring
- **Local inference**: hardware-controlled and without per-token API charges, but resource-limited

This comparison is intended as a deployment-oriented case study, not as a matched-capacity benchmark between equivalent model classes. Since Gemini 2.5 Flash and Qwen 2.5 1.5B differ in model size, serving infrastructure, and deployment environment, the observed differences should be interpreted as deployment-level outcomes arising from the combined effect of model capability, infrastructure optimization, and hardware budget, rather than as a pure API-versus-local causal effect.

B. System Architectures

Three representative LLM-based architectures were designed and evaluated:

- Prompt-Only
- Retrieval-Augmented Generation (RAG)
- Context-Optimized

Each system architecture was evaluated across both models. The architectures differ in how contextual information is constructed and supplied, while the underlying model type (API vs local) introduces additional differences in latency and cost characteristics.

1) Prompt-Only System: The prompt-only system embeds the entire regulatory document directly into the model prompt together with task instructions and the user query (Fig. 1). The model is explicitly instructed to extract the exact answer span from the provided text.

This architecture consists of a single inference step without preprocessing or external retrieval components [1], [2]. However, token usage grows linearly with document length and is bounded by the model's maximum context window [8]. No context filtering or compression is applied.

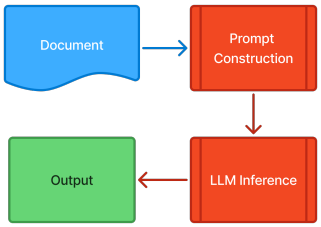

Fig. 1. Prompt-only system architecture.

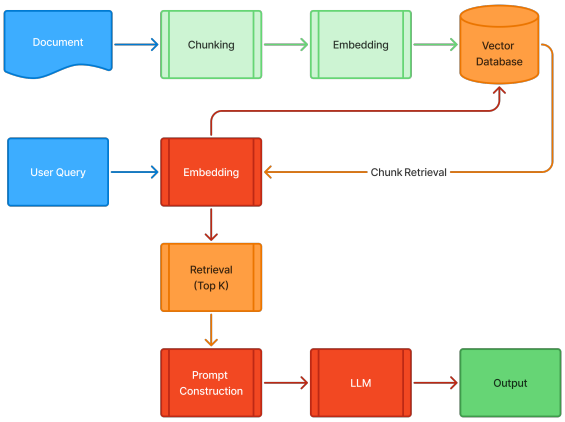

Fig. 2. Retrieval-augmented generation (RAG) system architecture.

2) Retrieval-Augmented Generation (RAG): The RAG system follows a two-stage retrieval–generation pipeline (Fig. 2) [3], [4].

a) Document Chunking: Each regulatory document was segmented into overlapping 256-token chunks to mitigate boundary information loss while preserving article-level semantic coherence. [3], [14].

b) Embedding Model: Chunks were embedded using the `sentence-transformers/all-MiniLM-L6-v2` model, a lightweight 384-D mode, for its favourable efficiency-quality trade-off.

c) Vector Indexing and Retrieval: Embeddings were indexed using FAISS with L2 similarity search. At inference time, the user query was embedded and used to retrieve the top-$k = 3$ most similar chunks, wchih were concatenated with the task prompt for answer extraction.

While RAG reduces irrelevant context exposure, it introduces additional components including embedding computation and similarity search, which contribute to latency and system complexity [5], [6].

3) Context-Optimized System: The context-optimized system is designed as a lightweight alternative positioned between

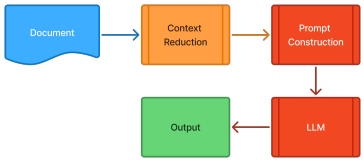

Fig. 3. Context-optimized system architecture.

prompt-only and RAG-based approaches (Fig. 3).

Instead of performing external retrieval, the full document is first segmented into chunks and scored using embedding similarity with respect to the query. All chunks are ranked globally, and only the highest-scoring segments within a predefined token budget are selected and concatenated into a structured context block supplied to the LLM.

This method should be viewed as a simple engineering baseline rather than a novel retrieval algorithm; the contribution of this study lies in its comparative evaluation on long, structured texts under deployment constraints. Unlike standard top-k RAG, which retrieves a small fixed number of chunks for generation, this approach performs global relevance ranking across all chunks and constructs a context under a token budget, thereby preserving broader document coherence while filtering irrelevant sections.

a) Design Rationale: EU regulatory texts such as GDPR, the AI Act, the DSA, the DMA, and the Data Governance Act are long, hierarchically structured, and dense in cross-references. Naively inserting full documents introduces noise and token inefficiency, while standard RAG may fragment logically connected provisions.

Recent work shows that long-context LLMs do not utilize all tokens uniformly and may suffer from attention degradation in the middle of long inputs [8]. The context-optimized design therefore aims to balance token efficiency, semantic relevance, document coherence, and reduced hallucination risk

C. Regulatory Document Corpus

The evaluation corpus consists of five long-form European Union regulatory documents:

- General Data Protection Regulation (GDPR) [9]
- Artificial Intelligence Act [10]
- Digital Services Act (DSA) [11]
- Digital Markets Act (DMA) [12]
- Data Governance Act [13]

These documents were selected due to their length, hierarchical structure, dense legal language, and cross-referencing, which present significant long-context reasoning challenges.

Each document was converted into structured plain text while preserving article numbering and paragraph hierarchy.

A dataset of 100 manually annotated extractive question-answer pairs (approximately 20 per document) was constructed. Questions cover definitions, obligations, prohibitions, numerical thresholds, and cross-references. Ground-truth answers were annotated as exact spans within the source documents.

D. Evaluation Metrics

The primary objective of this study is efficiency analysis. Therefore, token cost and latency are treated as primary metrics.

Token Usage: For each query, total token consumption is defined as

$$Tokens_{total} = Tokens_{input} + Tokens_{output} \tag{1}$$

Since transformer inference complexity scales quadratically with input sequence length due to self-attention mechanisms [15], prompt length directly impacts computational cost.

In all reported results, token usage refers to the observed total tokens per query, measured as the sum of realized input and output tokens returned by the system, rather than only the configured token budget. The values reported in the summary tables are averages over all 100 question-answer pairs.

Latency: End-to-end latency is measured using wall-clock timing:

$$T_{total} = T_{preprocess} + T_{retrieval} + T_{generation} \tag{2}$$

Latency measurement includes preprocessing, embedding computation (for RAG), vector retrieval, and language model generation. End-to-end measurement is standard in retrieval system evaluation [14].

Exact Match (EM): Since the task is formulated as extractive question answering, answer correctness is evaluated using Exact Match (EM), where a prediction is counted as correct only if it matches the annotated answer span exactly. EM is therefore a strict measure of extractive fidelity and may penalize semantically correct paraphrases or near-matches. [16].

E. Hardware Configuration

The local LLM (Qwen 2.5 Instruct) was executed on a system equipped with an NVIDIA RTX 4050 laptop GPU (6 GB VRAM), an Intel i7-13620H CPU, and 16 GB RAM.

The API-based model (Gemini 2.5 Flash) was accessed via a cloud service and therefore did not rely on local hardware for inference. However, end-to-end latency measurements include network communication overhead in addition to model response time.

All latency measurements were performed under stable network conditions to ensure consistency.

F. Experimental Hypotheses

The following hypotheses are evaluated:

- H1: API-based models achieve lower latency due to infrastructure optimization but incur monetary cost per query.
- H2: Local models exhibit higher latency but eliminate per-query cost.

TABLE I
QWEN: SUMMARY PERFORMANCE ACROSS ALL REGULATIONS

System	EM (%)	Avg. Latency (s)	Avg. Total Tokens
Context-Optimized	59.0	4.69	2521.95
Prompt-Only	0.0	67.02	4096.00
RAG	73.0	3.32	785.17

TABLE II
GEMINI: SUMMARY PERFORMANCE ACROSS ALL REGULATIONS

System	EM (%)	Avg. Latency (s)	Avg. Total Tokens
Context-Optimized	80.0	4.87	45264.56
Prompt-Only	80.0	5.68	79040.11
RAG	67.0	3.02	2665.14

- H3: RAG reduces token usage and may introduce retrieval overhead, but this overhead can be offset by shorter generation time.
- H4: Context-optimized processing provides the best cost–latency tradeoff across both deployment settings.

III. EXPERIMENTAL RESULTS

This chapter presents the empirical evaluation of the three system architectures across the regulatory corpus for both the local Qwen model and the API-based Gemini model. The analysis focuses on token usage, latency, and answer correctness, and examines the cost-latency trade-offs implied by these measurements.

A. Overall Performance Comparison

1) Qwen (Local Model): Table I summarizes the performance of the three systems when using the locally hosted Qwen 2.5 1.5B Instruct model.

The prompt-only system failed to produce any exact matches under the Qwen setup, primarily due to hard token limits and hardware constraints which prevented full document prompts from being processed. In practice, many queries approached or exceeded the maximum context window and output token limit, causing truncation and incomplete reasoning. By contrast, both RAG and context-optimized systems substantially reduced input length and achieved non-zero Exact Match scores, with RAG reaching the highest EM (73%) and the lowest average latency (3.32 s).

The context-optimized system achieved moderate EM (59%) with considerably lower latency than prompt-only and substantially fewer tokens than a full-document prompt. Qualitatively, RAG answers often captured the correct meaning of the target span (e.g., paraphrased obligations or conditions) even when they failed the exact wording required by EM, while the context-optimized system more often reproduced the precise extractive span due to the inclusion of larger coherent sections from the originating articles. A summary plot of EM versus latency for Qwen (Figure 4) further illustrates the dominance of RAG and context-optimized approaches over prompt-only in the local deployment regime.

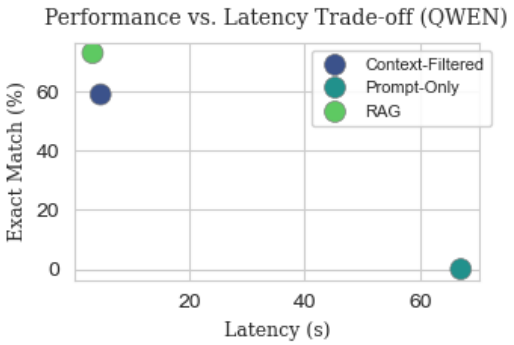

Fig. 4. Qwen: EM vs. latency across system architectures.

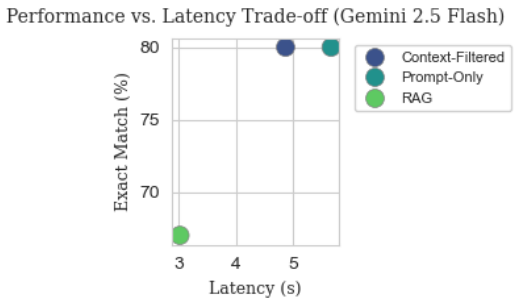

Fig. 5. Gemini: EM vs. latency across system architectures.

2) Gemini (API-Based Model): Table II reports the corresponding results for the Gemini 2.5 Flash API-based model.

For Gemini, both context-optimized and prompt-only systems achieved the same average EM of 80%, indicating that the API-based model can handle very long prompts more robustly within its larger context and output limits. However, this accuracy parity hides substantial differences in efficiency: prompt-only required on average almost 80k tokens per query, while context-optimized reduced token usage by more than 40% and slightly improved latency.

The RAG system again achieved the lowest token usage and the fastest average latency (3.02 s), but with lower EM (67%) under the strict extractive metric used in this study. As with Qwen, RAG answers were often semantically close to the ground-truth span yet failed the exact-span EM criterion.

B. Prompt-Only Analysis

For the Qwen deployment, prompt-only performance was dominated by context window and compute constraints. Long EU regulations easily approach or exceed several thousand tokens when entire articles or recitals are included, and the 32k-token context window combined with a relatively small GPU (6 GB VRAM) forced aggressive truncation of inputs and outputs. As a result, the model often saw only partial documents and failed to include the answer span within the visible context. This explains the observed 0% EM despite very high average latency (67.02 s), as Qwen had to process near-maximum-length inputs under limited hardware.

By contrast, Gemini's managed infrastructure allowed prompt-only inputs to include much larger portions of the regulatory texts without hitting hard context or output limits, leading to substantially higher EM (80%) at a still acceptable latency of 5.68 s on average. Nevertheless, the token counts (around 79k tokens per query) imply significant monetary cost and raise concerns about scalability for production deployments where millions of queries may be processed monthly.

Across documents, Prompt-Only performed better on shorter or simpler texts and worse on long, highly cross-referenced regulations where relevant provisions could not always be included in one prompt window.

C. Retrieval Performance (RAG)

For both models, RAG substantially reduced average token usage by restricting the context to the top-3 retrieved chunks. This reduction was most pronounced for Gemini, where the average token count dropped from tens of thousands (prompt-only/context-optimized) to approximately 2,665 tokens per query, with a concurrent reduction in latency to around 3 s.

Retrieval Recall@5 was generally high for questions whose answers were localized within a single article or a small cluster of adjacent paragraphs. In these cases, the correct chunk was often retrieved, and the model produced answers that were semantically faithful to the ground-truth span. However, RAG struggled with questions requiring dispersed evidence (e.g., definitions located in one article and conditions in another) or cross-references spanning multiple titles or chapters, which is common in EU legislation [9], [11]–[13]. In such cases, Recall@5 dropped, and the answer either missed critical conditions or hallucinated unsupported constraints, consistent with prior observations about retrieval failures in knowledge-intensive tasks [3], [14].

Even when retrieval succeeded, EM sometimes remained 0 because the generator produced paraphrased explanations rather than verbatim spans. Since EM is a strict extractive metric [16], such paraphrases were scored as incorrect despite being semantically correct or practically useful in legal question answering.

D. Context-Optimized System Analysis

The context-optimized system lies between prompt-only and RAG in both complexity and information access. It scores all chunks globally, then builds a context within a token budget while preserving larger contiguous spans of the regulation to reduce attention dilution in long inputs [8].

With Qwen, it reached 59% EM at 4.69,s and about 2,522 tokens, versus 0% EM and 67.02,s for prompt-only, but still below RAG (73% EM, 785 tokens, 3.32,s). For Gemini, it achieved the best EM (80%), matching prompt-only but with lower latency (4.87,s vs. 5.68,s) and far fewer tokens (about 45k vs. 79k), while outperforming RAG on EM (67%).

Qualitatively, it tends to include entire articles or related paragraphs, helping resolve cross-references and multi-clause conditions in EU regulations [9]–[13]. Overall, context optimization is particularly attractive in API settings where tokens are costly but long-context processing is available, offering a strong accuracy–efficiency trade-off and aligning with recent work on structured prompting for long legal documents [3], [7], [8].

E. Context Length Sensitivity

Across both deployment settings, latency scaled strongly with input token length, as expected from the quadratic complexity of self-attention with respect to sequence length [15]. For Qwen, this effect was amplified by GPU memory limits: near-maximal context lengths led to frequent paging and slower generation, which is reflected in the 67 s average latency for prompt-only versus 3–5 s for RAG and 4–7 s for context-optimized systems. For Gemini, latency remained within a relatively narrow band (3–6 s) despite large variation in token counts, although shorter prompts still yielded faster responses.

Extremely short contexts (e.g., under-retrieved RAG chunks) lacked sufficient information and led to hallucinations or partial answers. Conversely, excessively long contexts (full documents in prompt-only) sometimes diluted attention around the relevant span, especially in the local model, consistent with prior work on long-context degradation [8].

F. Cost-Latency Comparison

The measured token usage and latency can be used to reason about operational cost. For the API-based Gemini model, monetary cost scales approximately linearly with the number of input and output tokens, with per-token pricing set by the provider.[1] Under this model, the prompt-only system is the most expensive, consuming roughly 79k tokens per query. The context-optimized system reduces token usage to about 45k tokens, corresponding to an approximate 40% cost reduction while maintaining the same EM as prompt-only in the Gemini setting. RAG further reduces tokens to roughly 2.7k per query, representing an order-of-magnitude cost reduction at the expense of lower EM.

In contrast, the local Qwen deployment has near-zero marginal cost per additional token once hardware is provisioned, but latency and feasibility are constrained by GPU memory and throughput. Here, the prompt-only system is not just slow but effectively unusable due to timeouts and context truncation, while RAG and context-optimized systems deliver acceptable latency (3–5 s) and non-zero EM without additional monetary cost. This aligns with broader observations that local deployments favor architectures that aggressively control context length in order to fit hardware budgets, whereas API-based deployments must balance per-token cost against the benefits of longer context [3], [7], [14].

From a cost-latency perspective, RAG occupies the "low-cost/low-latency but medium-accuracy" corner, while prompt-only sits at "high-cost/high-latency but potentially high-accuracy" (for sufficiently capable APIs), and context-optimized lies in a middle regime that offers competitive ac-

[1] Exact pricing is omitted here for generality; practitioners can plug in their own cost schedule.

curacy with moderate cost and latency across both deployment settings.

IV. Discussion

A. Simplicity vs. Efficiency

From a cost-latency perspective, RAG occupies the "low-cost/low-latency" corner but with lower EM under strict extractive evaluation, while prompt-only sits at "high-cost/high-token usage" and remains feasible mainly for sufficiently capable API models. The context-optimized system lies in a middle regime that offers competitive extractive performance with substantially lower token usage than full-document prompting. [3]–[5].

B. Long Context vs. Selection

The results indicate that simply increasing context length is not sufficient. Long prompts can dilute attention and waste tokens, especially on dense legal texts [8]. RAG aggressively reduces context, trading some recall for speed and cost, whereas context optimization strikes a middle ground: fewer tokens than full documents, but more coherent spans than a small set of local chunks, which is beneficial for cross-referential texts [9], [11]–[13].

C. Deployment Considerations

For local deployments, RAG or context-optimized variants are effectively mandatory in the tested hardware setting, as naive long-context prompting is too slow or fails. For API deployments, context optimization appears attractive when exact answers and traceability matter, while RAG is preferred for low-cost, low-latency scenarios where small drops in EM are acceptable.

This observation should be interpreted with care: the local results reflect a small 1.5B model running on a 6 GB laptop GPU, and should not be generalized to larger local deployments with server-grade hardware or higher-capacity open-weight models.

V. Limitations

This study has several limitations. First, it is not a matched-capacity benchmark between equivalent API and local models. Gemini 2.5 Flash and Qwen 2.5 1.5B differ in model size, serving stack, and deployment environment, so the reported differences should be interpreted as deployment-level outcomes rather than as a pure API-versus-local causal effect. Second, the local setup is intentionally resource-constrained, using a laptop GPU with 6 GB VRAM; the results therefore should not be generalized to larger local deployments using 7B–70B models or server-class hardware. Third, answer quality was evaluated primarily using Exact Match, which is appropriate for extractive QA but systematically penalizes semantically correct paraphrases and near-matches, especially in RAG outputs. Accordingly, the reported EM values should be interpreted as a measure of strict extractive fidelity rather than a complete measure of semantic usefulness or legal adequacy. Finally, the study focuses on extractive QA over EU regulatory texts, and the conclusions may not transfer directly to other tasks such as summarization, open-ended compliance reasoning, or multi-document synthesis.

VI. Future Work

Future work includes evaluating larger local models and specialized long-context architectures, and combining RAG with context optimization under a joint token budget. A particularly important next step is a matched-capacity comparison between local and API-served models to disentangle deployment effects from differences in model scale and capability. More realistic cost models (including batching and multi-tenant workloads) and broader tasks such as summarization and scenario-based compliance analysis should be explored. Better human expert evaluation could complement EM to better capture the usefulness of near-exact or paraphrased answers in legal settings.

VII. Conclusion

This work empirically tested the four hypotheses outlined in Section II.F across Qwen (local) and Gemini (API) deployments on regulatory QA.

H1: "API-based models achieve lower latency due to infrastructure optimization but incur monetary cost per query."
Supported in the tested setup. Gemini consistently achieved lower latency in the API setting, while also incurring per-token usage costs that become substantial at large prompt sizes.

H2: "Local models exhibit higher latency but eliminate per-query cost."
Supported. Qwen eliminated monetary cost but suffered high latency (67 s for prompt-only) due to GPU constraints, while RAG and context optimization reduced it to 3.5 s.

H3: "RAG reduces token usage but introduces additional latency regardless of model type."
Supported in the tested setup. The local Qwen deployment eliminated per-query API cost but showed stronger sensitivity to context length and hardware limits, especially for prompt-only inference.

H4: "Context-optimized processing provides the best cost–latency tradeoff across both deployment settings."
Partially supported. For Gemini, Context-Optimized matched Prompt-Only EM (80%) with approximately 40% fewer total tokens and slightly lower latency. For Qwen, Context-Optimized substantially improved over unusable Prompt-Only performance (0% EM) but was outperformed by RAG in both latency and EM.

Overall, no single architecture universally dominates. RAG is most attractive when low latency and low token usage are the primary objectives, while Context-Optimized offers a stronger balance between strict extractive performance and efficiency in the API setting. Prompt-Only remains the simplest architecture but is rarely the most practical for long regulatory texts. These findings should be interpreted as deployment-level outcomes under the tested model and hardware configurations, rather than as a general claim that API inference is inherently superior to local inference.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al., "Language models are few-shot learners," in *Proc. 34th Int. Conf. Neural Information Processing Systems (NIPS'20)*, Dec. 2020, Art.no. 159, pp. 1877–1901.

[2] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," *ACM Computing Surveys*, Sep. 2022, vol. 55, no. 9, pp. 1–35.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. 34th Int. Conf. Neural Information Processing Systems (NIPS'20)*, Dec. 2020, Art. no. 793, pp. 9459–9474.

[4] Y. Gao et al. et al., "Retrieval-Augmented Generation for Large Language Models: A survey," *arXiv (Cornell University)*, Dec. 2023, doi: 10.48550/arxiv.2312.10997.

[5] S. Ray et al., "METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation," *arXiv (Cornell University)*, Dec. 2024, doi: https://doi.org/10.48550/arxiv.2412.10543.

[6] W. Jiang, S. Zhang, B. Han, J. Wang, B. Wang, and T. Kraska, et al., "PipeRAG: Fast Retrieval-Augmented Generation via Algorithm-System Co-design," *arXiv (Cornell University)*, Mar. 2024, doi: 10.48550/arxiv.2403.05676.

[7] Z. Li, C. Li, M. Zhang, Q. Mei, and M. Bendersky, "Retrieval Augmented Generation or Long-Context LLMs? a comprehensive study and hybrid approach," *arXiv (Cornell University)*, Jul. 2024, doi: 10.48550/arxiv.2407.16833.

[8] N. F. Liu, "Lost in the Middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, Jan. 2024, doi: 10.1162/tacl_a_00638.

[9] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)," *Official Journal of the European Union*, L119, pp. 1–88, Apr. 2016. http://data.europa.eu/eli/reg/2016/679/oj

[10] European Parliament and Council of the European Union, "Regulation of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," 2024. http://data.europa.eu/eli/reg/2024/1689/oj

[11] European Parliament and Council of the European Union, "Regulation (EU) 2022/2065 on a single market for digital services (Digital Services Act)," *Official Journal of the European Union*, L277, pp. 1–102, Oct. 2022. http://data.europa.eu/eli/reg/2022/2065/oj

[12] European Parliament and Council of the European Union, "Regulation (EU) 2022/1925 on contestable and fair markets in the digital sector (Digital Markets Act)," *Official Journal of the European Union*, L265, pp. 1–66, Oct. 2022. http://data.europa.eu/eli/reg/2022/1925/oj

[13] European Parliament and Council of the European Union, "Regulation (EU) 2022/868 of the European Parliament and of the Council of 30 May 2022 on European data governance and amending Regulation (EU) 2018/1724 (Data Governance Act)," *Official Journal of the European Union*, L152, pp. 1–45, Jul. 2022. http://data.europa.eu/eli/reg/2022/868/oj

[14] N. Thakur, N. Reimers, A. Rücklé, Abhishek Kumar Srivastava, and Iryna Gurevych, "BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models," Apr. 2021, doi: https://doi.org/10.48550/arxiv.2104.08663.

[15] A. Vaswani et al., "Attention Is All You Need," *arXiv.org*, 2017. https://arxiv.org/abs/1706.03762

[16] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," *ACLWeb*, Nov. 01, 2016. https://www.aclweb.org/anthology/D16-1264/

Appendix 2: Code and Data Availability

The code, experimental setup, datasets, and processing scripts used in this study are available in the author's public repository:

GitHub Repository:
https://tinyurl.com/AIENGSEM

Appendix 2: Rebuttal

Rebuttal to the reviewers comments can be found through the link.

Rebuttal Document:
https://tinyurl.com/rebuttalAIEng